Algoritmos de Sistemas Distribuídos

Guilherme Henrique Ferreira da Silva
260469

Kairé Pereira Giovanetti
260558

I. Resumo

O projeto consistiu na implementação de diferentes algoritmos aplicados em Sistemas Distribuídos, visando promover escalabilidade, eficiência e disponibilidade. Para que fosse possível dimensionar e visualizar um sistema na prática, simulou-se um cenário real através de um ambiente em nuvem utiizando quatro Máquinas Virtuais (VMs) distribuídas em regiões geográficas distintas da plataforma Google Cloud Platform (GCP).

Os algoritmos escolhidos, muito comuns em Sistemas Distribuídos, foram: Chord, protocolo baseado em Distributed Hash Table (DHT), regularmente usado em Sistemas Peer To Peer (P2P); Primary-Backup Replication, que busca garantir a durabilidade de dados; Bully, algoritmo de eleição para coordenação e determinação de liderança.

Desse modo, buscou-se mostrar e comprovar a robustez do Sistema promovida por tais algoritmos. Para isso, exploraram-se cenários que explicitam a tolerância à falhas e desempenho eficiente da Arquitetura.

O código está disponível em https://gitlab.ic.unicamp.br/mc714-2025-2/group_2/projeto2, e o vídeo em link do vídeo.

II. Contextualização

Analizando o contexto atual, onde há a necessidade de grande escalabilidade de sistemas e recursos, assim como transferências de dados em grande volume, o objetivo central, dada a proposta do projeto, é solucionar problemas como o roteamento eficiente e a garantia da durabilidade de dados.

Tendo isso em mente, decidiu-se abordar os seguintes algoritmos:

- Chord: protocolo que permite o particionamento de chaves e roteamento P2P, garantindo complexidade O(log n) para a consulta e busca de chaves.
- Primary-Backup Replication: integrado ao Chord, permite a recuperação de dados do sistema no caso de queda de nós, demonstrando tolerância à falhas.
- Bully: implementado separadamente para estudar os conceitos de coordenação e autonomia dos nós, que conseguem determinar um líder de maneira autônoma.

III. Ambiente de Execução

Utilizando seriviços da GCP, o sistema foi construído em ambiente em nuvem e, assim, possibilitando a simulação das lateências e complexidade de um sistema real e de comunicação global. Foram instanciadas quatro VMs em regiões geográficas distintas ($asia - east$, $europe - west$ e $us-central$), com configurações e2-micro (2 vCPUs, 1 GB de memória) e sistema operacional Ubuntu 24.04. A comunicação entre os nós é viabilizada por uma regra de Firewall, que permite transmissões TCP nas portas escolhidas (50051, por exemplo) para todos os IPs externos.

IV. Camada de Comunicação

A comunicação entre os nós se deu por chamadas de procedimento remoto (RPCs) através do framework Google Remote Procedure Cal (GRPC). O GRPC usa buffers de protocolo (Linguagem de Definição de Interface (IDL), em arquivos .proto) para estabelecer mensagens e definir serviços, que garantem a interoperabilidade dos nós.

Para o Chord, foram definidas as seguintes RPCs:

- Put: Insere uma chave no anel.
- Get: Recupera o valor armazenado em alguma chave em algum nó do anel
- Notify: Notifica o seu sucessor, informando que é o seu predecessor.
- Lookup: Encontra o nó responsável por uma chave ou ID.
- GetPredecessor: Recupera as informações do predecessor do nó.
- GetSucessor: Recuperas as informaçõe do sucessor do nó.

Já para o Bully, as RPCs definidas foram:

- AnunciaEleição: Anuncia para um nó que uma eleição foi iniciada.
- SetLider: Afirma para um nó que um novo líder foi estabelecido.
- HealthCheck: Confere se o líder continua vivo.

Para o Consenso foram definidas:

- PerguntaNumero: Descobre o valor de ”valor” quando um nó é iniciado.
- RecebePropostaNumero: Recebe a proposta de um novo valor para ”valor.
- PerguntaOpiniao: Pergunta para os nós se eles aceitam o novo valor proposto.
- AfirmaNumero: Afirma o novo valor proposto após a votação.

Por fim, para a Replicação, definiu-se uma única RPC a mais:

- PutReplica: Insere uma réplica da chave no sucessor do nó.

Cada VM (nó) é inicializada como um servidor GRPC e gera Stubs para possibilitar a comunicação entre nós e a manutenção do Sistema P2P. No caso dos algortimos de Chord e Replicação, implementados em conjunto, depende-se da existência e determinação de um nó de $bootstrap$ (primeiro nó a se conectar à rede).

V. Chord e Replicação

Dadas as RPCs e o objetivo do projeto, então, o Chord e o algoritmo de Primary-Backup Replication foram implementados de modo que se completassem, unindo o roteamento eficiente e a tolerância a falhas.

A. Distributed Hash Table

A rede de sobreposição do Sistema P2P implementado se baseia no Chord, que é uma tabela hash distribuída, com topologia em forma de anel, e atribui pares "chave-valor" para diferentes nós, cuja responsabilidade sobre as chaves se dá pelo seu próprio ID.

O Chord se baseia no hashing consistente (algoritimo SHA-1), com os nós e chaves sendo designados com IDs de m bits e formando, então, um anel com até 2^m nós. A chaves são armazenadas nos nós de modo que uma chave k é salva no nó com o menor identificar $id >= k$, assim como mostra a Figura 1.

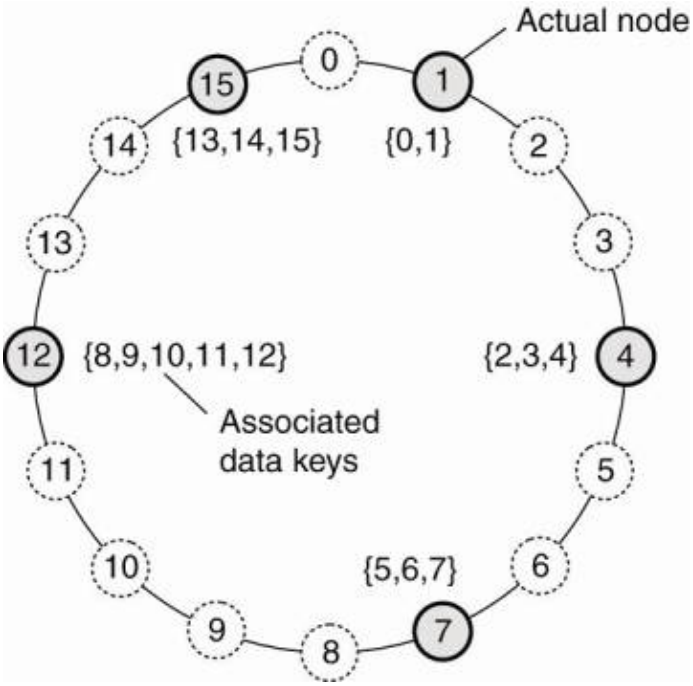

Fig. 1: Estrutura do anel Chord.

B. Lookup e Finger Table

Para que arquitetura funcione corretamente, cada nó tem que possuir as informações, no mínimo, de seu predecessor e sucessor imediatos. Desse modo, é possível obter a funcionalidade básica do Chord de roteamento e busca de chaves com a função $Lookup(k)$. Com apenas a informação do sucessor, seria possível realizar uma $query$ em tempo linear ($O(n)$) passando por todos os sucessores até encontrar o nó responsável pela chave. Nota-se que a função de Lookup é importante também para a inserção dos nós no anel e inserção das chaves (com a função $Put()$).

Para melhorar o roteamento, a implementação do Chord requer uma tabela adicional Finger Table de m entradas. Cada entrada i na Finger Table é definida por

$$(n - 2^{i-1}) \bmod 2^m, \tag{1}$$

onde n é o ID do nó em questão. É possível visualizar um exemplo de Finger Table na Figura 2.

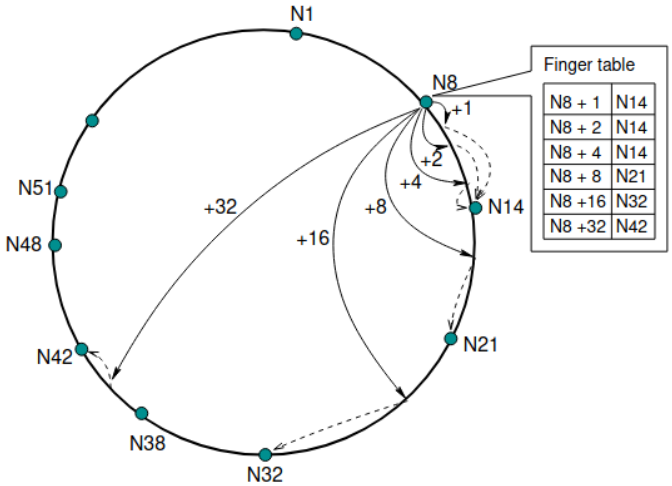

Fig. 2: Exemplo de Finger Table para nó N8.

Desse modo, ao fazer o $Lookup(k)$, com a Finger Table estabelecida, itera-se sobre ela a partir da maior entrada i, de modo que se checa todo o intervalo entre o nó e o respectivo nó de i. Caso a chave k esteja dentro desse intervalo, a partir da regra de responsabilidade de cada nó, a iteração continua para $i - 1$. Caso não esteja, é feita uma nova chamada de $Lookup(k)$ para o nó da entrada i, funcionando como um atalho para encontrar a chave k. Uma vez que as entradas da Finger Table representam nós com distâncias de potências de 2, essa estratégia garante que o Lookup tenha complexidade $O(Log\ n)$. Na Figura 3, é possível visualizar uma chamada de $Lookup(k)$ para a chave 54 no nó 8, exemplificando os processos de atalhos explicados acima.

C. Integridade dos Nós e atualização da Finger Table

É necessário estabelecer métodos de estabilização que garantam a consistência e integridade do anel, mantendo a estabilidade do Sistema em caso de falha, desconexão, ou entrada de algum nó. Para isso, criou-se uma thread separada para ser executada em paralelo e garantir que os nós possam atualizar de maneira autônoma. Essa thread, então, para cada nó, é responável por verificar se houve a inserção de algum nó

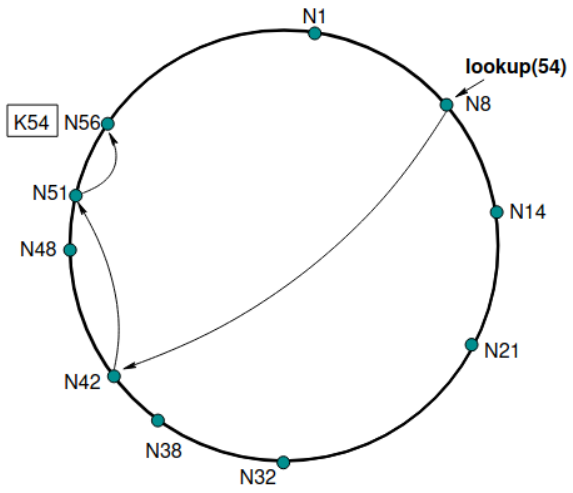

Fig. 3: Chamada de Lookup(54) para nó N8.

entre ele e seu sucessor, atualizando os campos de predecessor e sucessor dos involvidos se necessário, ou verificar se o seu sucessor imediato foi desconectado . Isso é feito, além da função $GetPredecessor()$, por meio da função $Notify()$, que notifica o seu sucessor imediato que o nó em questão é o seu predecessor.

Além disso, é também preciso atualizadr a Finger Table de acordo com a entrada e remoção de nós. Também foi criada uma thread para ser executada em paralelo, e isso é feito basicamente através da chamada de Lookup() com os valores calculados com a Equação 1.

D. Replicação e Tolerância a falhas

Para garantir a alta disponibilidade do Sistema e a durabilidade dos dados, foi implementado, em conjunto com o Chord, o algoritmo de Primary-Backup Replication. Dessa maneira, busca-se replicar as chaves em outros nós do anel e, assim, caso um nó seja desconectado, o dado ainda possa ser acessado. Nesse caso, decidiu-se replicar a chave no sucessor imediato, uma vez que esse já seria o responsável pela chave em uma suposta desconexão do nó. Desse modo, a função $Put()$ deve, além de inserir uma chave em um nó, chamar a RPC $PutReplica()$ para adicionar a réplica em seu sucessor. A função $Get()$, por sua vez, caso não encontre a chave no nó retornado por $Lookup()$, também deve fazer a busca em seu sucessor.

Além disso, na implementação feita, cada nó possui conhecimento de seus dois sucessores, garantindo maior estabilidade ao Peer Churn. Desse modo, com a saída de nós dos sistema, é garantido uma maior consistência do anel, já que os sucessores são atualizados corretamente. Para aumentar essa tolerância, seria possível ainda aumentar a lista de sucessores conhecidos, melhorando também o Primary-Backup Replication.

E. Experimentação e análise dos resultados

Por fim, é necessário validar a implementação no ambiente em nuvem GPC usando as quatro VMs. Dessa forma, a fim de evidenciar os resultos no relatório, além do que é apresentado no vídeo de apresentação de projeto, por meio de um código simples de simulação de um cliente, criar cenários para testar o $Lookup(k)$ e sua complexidade e a replicação de chaves nos sucessores. Tal código, representando um cliente, possibilita a inserção e recuperação de chaves, assim como a checagem do sucessor e predecessor de cada nó

Assim, na Figura 4, mostra-se o log do uso da Finger Table para encontrar o nó possuidor de uma chave. O nó que exibiu o log (IP 104.199.95.134, localizado na Europa) estava a uma distância de dois nós do nó procurado (IP 35.221.134.25, localizado na Àsia), mas como este se apresentava no indíce 158 de sua Finger Table, houve o salto direto, ao invés de passar por seu sucessor (o que caracterizaria uma busca linear). Desse modo, apesar de ser um ambiente com poucos nós, é evidente que o $Lookup(k)$ funciona como esperado e apresenta tempo de busca logaritmico.

```
LOOKUP-FINGER: Usando atalho índice 158 para 50051
FINGER: Saltando para 50051
DEBUG: Nó possuidor da chave está no endereço 35.221.134.25:50051
```

Fig. 4: Log de teste da Finger Table do anel Chord.

De maneira semelhante, testou-se a remoção de um nó, responsável por alguma chave, checando se a ideia da replicação nos sucessores estava sendo aplicada. Assim, como mostra a Figura 5, após a remoção do nó da Europa, foi feita uma query de busca para a chave ”chave-remocao”, que foi encontra com sucesso no nó da Ásia.

```
DEBUG: Nó possuidor da chave está no endereço 35.221.134.25:50051

Operação: (get/put/status)get
IP: 104.199.95.134
Porta: 50051

Chave: chave-remocao
* Valor da chave 'chave-remocao' encontrado com sucesso: Valor de t
este para testar Replicacao de Chaves!
```

Fig. 5: Log e output de teste da replicação nos sucessores.

Outros testes, tais como a checagem da atualização de sucessores e predecessores foram feitos (alguns deles mostrados no vídeo do projeto), e os algoritmos implementados apresentaram resultados consistentes e de acordo com o exigido.

VI. Bully e Consenso

A. Bully

O algoritmo do Bully consiste em um protocolo para determinar um líder em uma rede na qual todos os nós tem conhecimento de todos os outros nós, caso o líder anterior seja detectado fora do ar ou a rede tenha acabado de ser iniciada. A ideia central do algoritmo Bully se baseia na existência de id’s crescentes para cada nó, de modo que o nó com o id mais alto deve ser eleito o líder.

O procedimento funciona da seguinte maneira: quando um nó percebe que o líder anterior da rede não está respondendo,

ou pela função $HealthCheck(mensagemNumero)$, que tem o trabalho de conferir se o líder está vivo, ou por alguma outra função de comunicação com o líder, ele inicia um processo de eleição com a função $ComecaEleicao()$. Nesse processo, ele envia um mensagem de eleição para todos os nós da rede com id mais alto que ele utilizando a função $AnunciaEleicao(mensagemNumero)$, ademais, são usadas a função auxiliar $_PreparaMensagemEleicao(id_destino)$, e a biblioteca $futures$ para garantir que essas mensagens sejam enviadas ao mesmo tempo, evitando que o nó caia no meio do processo. Se nenhum desses nós responder isso quer dizer que ele é o nó com id mais alto ativo na rede nesse momento, e então ele se declara o líder, e anuncia isso para todos os outros nós da rede utilizando a função $AnunciaLider()$, reponsável por chamar a função $_PreparaMensagemLider(id_destino)$, para paralelizar os envios, que por fim realiza a comunicação de fato utilizando a função $SetLider(mensagemVitoria)$, que também é responsável por garantir que o "valor" de todos os nós sejam iguais. Caso algum nó mande uma mensagem contestando a eleição ele interrompe seu processo de eleição, e o nó que respondeu inicia o seu próprio processo, que funciona de modo igual ao original. Isso se repetirá para todos os nós até que apenas o de id mais alto ativo na rede naquele momento não seja contestado, permitindo que ele assuma a posição de líder.

Já para nós novos que são reinseridos na rede, inicialmente ele acessa o "valor" de outros nós, e o copia para si, e após isso inicia uma eleição.

Para exemplificar esse procedimento, considere um processo com 5 nós em que o líder anterior, o nó 5, se desconecta da rede e o nó 2 note sua ausência. Isso fará com que o nó 2 inicie uma eleição, mandando uma mensagem para todos os nós com id maior que o dele (Fig. 6). Então, os nós 3 e 4 que ainda estão vivos irão responder ao nó 2, que irá encerrar seu processo de eleição (Fig. 7). Os nós 3 e 4 iniciam suas próprias eleições (Fig. 8), e o processo se repete então até que o nó 4 se declare como líder (Fig. 9). Note que, nesse ponto, como o nó 5 está desconectado, o nó 4 é de fato o com maior id, e a eleição ocorreu corretamente.

B. Consenso

Para o algoritmo de consenso foi definida uma variável "valor", presente em todos os nós, que deve ser consistente através de todos eles, e pode receber sugestões de modificação por qualquer um dos nós. Essas sugestões, por sua vez, podem ser aceitas ou rejeitas, com base em uma votação. Para os efeitos de teste, não foi atribuído nenhum significado à variável nem ao voto de cada nó em uma deteminada votação, sendo eles decididos aleatóriamente. Apesar disso, o sistema funciona de modo idêntico a se esses valores e decisões tivessem algum significado, não sendo necessário realizar alterações no protocolo de consenso.

Para a realização do protocolo, existe um líder que está eleito pela utilização do Algoritmo do Bully, dessa forma, se em determinado momento um nó tem uma sugestão de mudança para o "valor", o seguinte é feito:

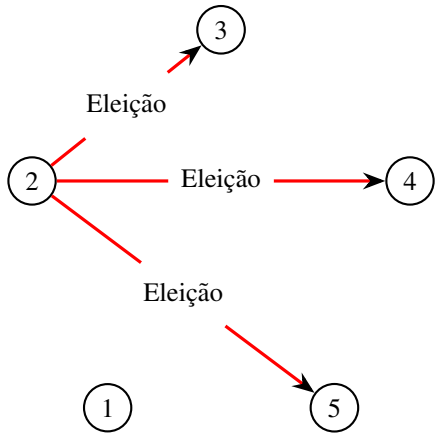

Fig. 6: Nó 2 inicia uma eleição.

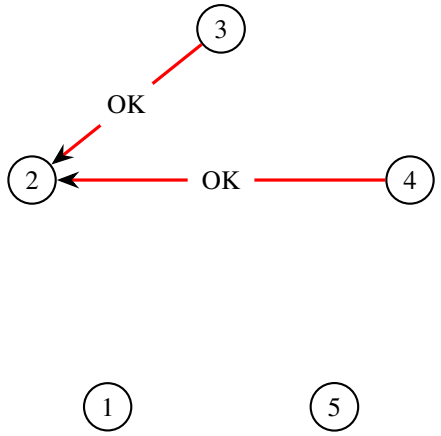

Fig. 7: Nós 3 e 4 respondem a eleição do nó 2.

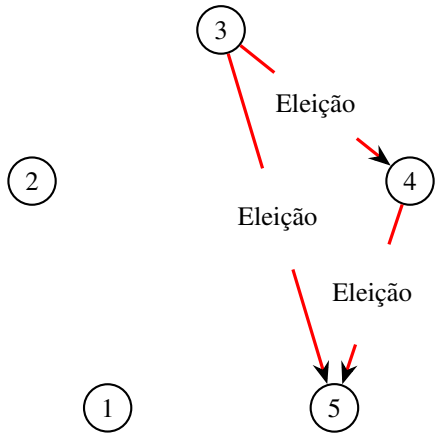

Fig. 8: Nós 3 e 4 começam suas próprias eleições.

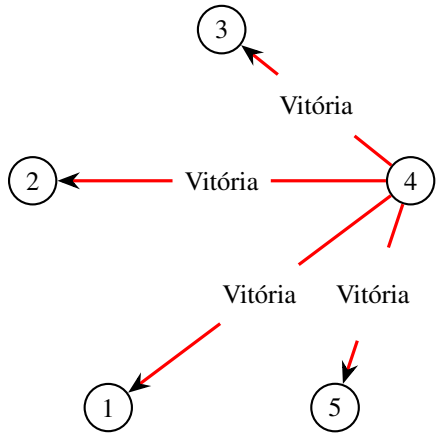

Fig. 9: Nó 4 se declara o líder e termina o processo de eleição.

o nó envia a sugestão para o líder através da função $RecebePropostaNumero(mensagemNumero)$, que ao recebê-la envia uma votação para todos os nós por meio da função $PerguntaOpiniao(mensagemNumero)$, e decide um voto próprio. Após receber as respostas dos nós, o líder analisa se metade ou mais são positivas, e caso sim, prepara as mensagens para afirmar o número com a função $_PreparaMensagemNumero(id_destino)$, e a biblioteca $futures$, responsáveis pela execução da função $AfirmaNumero(mensagemNumero)$ para todos os nós em conjunto.

C. Tolerância a Falhas

A tolerância a falhas do sistema é garantida pelos seguintes fatos: quanto à eleição de líder, o processo é anunciado para todos os nós em paralelo, dessa forma, caso o nó que está se elegendo caia no meio do processo nenhum nó receberá essa atualização. Dessa forma, para todos os nós que estão na rede o líder continua sendo o nó que originalmente se tornou indisponível, e, tanto se ele tiver voltado nesse intervalo, como se não, a rede permanece constante.

Já para o processo de consenso, também não há possibilidade de dessincronização, pois, tanto o processo de coleta de votos, como o de anúncio de resultados são feitos em paralelo. Para o processo de coleta de votos, como todos os votos são feitos em conjunto, o líder só começa a decisão após todos os nós vivos na rede nesse momento responderem, não havendo possibilidade de que votos relevantes sejam ignorados. Para a distribuição do novo valor, como todos os nós recebem ele ao mesmo tempo, de modo análogo ao anúncio do líder, não há possibilidade de dessincronização.

VII. Conclusão

Finalmente, concluí-se que a implementação dos algoritmos escolhidos foi satisfatória e bem-sucedida, uma vez que foi possível simular cenários reais e estudar o funcionamento na prática de Sistemas Distribuídos, gerando latências reais de comunicação e abordando situações complexas de roteamento.

Em específico para o Chord, foi possível gerar um protocolo de $Lookup(k)$ de complexidade logaritmica, assim como proposto pelo projeto, e gerar um sistema robusto tolerante a diversos tipos de falha. A robustez e consistência do sistema foi, ainda, impulsionada pela implementação do Primary-Backup Replication, que possibilitou uma maior disponibilidade e durabilidade dos dados.

Ademais, para o algoritmo do Bully, foi possível atingir uma rede na qual nós podem se desconectar e reconectar mantendo a consistência do líder eleito pela distribuição simultânea dos valores, sendo realizada, ainda, a implementação de um protocolo simples de consenso entre os nós, que mantém seus estados constantes.

VIII. Referências

1 Chord: A Scalable Peer-to-peer Lookup Protocol for Internet Applications
2 Tanenbaum, Andrew S, Van Steen, Maarten. Distributed systems: principles and paradigms. Pearson/Prentice Hall. 4th Ed.
3 Visão geral do gRPC
4 Criar uma instância de VM do Linux no Compute Engine